This tells us that a SimpleXMLElement object can be constructed with between one and five parameters (the first has no default value) to control initial state, it has public methods `addAttribute()`, `addChild()`, `asXML()`, and so on, and how to call those methods with the necessary parameters. The `implements Stringable, Countable, RecursiveIterator` portion of the declaration lists the interfaces provided by the class. This serves as a kind of contract to client code, interfaces guarantee that objects of this class will behave in certain ways.

It is also possible in PHP to formally declare interfaces in code, and to use them as type declarations.

Declaring Interfaces

An `interface` declaration in PHP is code that is very similar to a class declaration but serves to only define the methods a class must implement. Interfaces are defined in the same way as a class but with some caveats. An interface can only declare constants and method signatures (not implementation), they cannot declare properties. Additionally, only public methods and constants are declared. Any attempt to add properties or private members will result in fatal errors. Here's a working example of an interface:

```
interface QuoteBotInterface
{
    public function getQuote(): string;
}
```

This code does not produce any output; it simply exists to define how something that implements it will work. Here's how we can make use of it:

```
class QuoteBot implements QuoteBotInterface
{
    private array $quoteStash = [
        "Obviously, you're not a golfer." . PHP_EOL,
        "That's just, like, your opinion, man." . PHP_EOL,
        "Forget it, Donny You're out of your element." . PHP_EOL,
    ];

    public function getQuote(): string
    {
```